CSV – XML - JSON

Generell unterscheidet man zwischen drei verschiedenen Dateiformaten, die heutzutage eine Rolle spielen: CSV, XML, JSON
Als Beispiel verwenden wir folgende Informationen von Personen, die dargestellt werden sollen:



Thomas Becker,
45 Jahre alt, wohnt in Stuttgart,
Lieblingshobby: Wandern, Zweithobby: Kochen



Maria Müller,
32 Jahre alt, wohnt in Stuttgart
Lieblingshobby: Lesen, Zweithobby: Radfahren

CSV (**C**omma **s**eparated **V**alues)

```
1 Name,Alter,Stadt,Hobby1,Hobby1_Prio,Hobby2,Hobby2_Prio
2 Maria Müller,32,Stuttgart,Lesen,Prio1,Radfahren,Prio2
3 Thomas Becker,45,München,Wandern,Prio1,Kochen,Prio2
```

Zeile 1 kann die Daten genauer Beschreiben. Wenn eine Tabelle dargestellt werden soll, sind es meistens die Spaltenüberschriften. Diese Zeile kann auch entfallen.
Zeile 2-3 und weitere enthalten die Datensätze. In der Regel sind die Datensätze gleich lang und entsprechen immer einer Information. Sie entsprechen den Datensätzen einer Tabelle.
Dateiendung häufig „.csv“.

Trennung der Informationen: Die Informationen sind in der Regel durch **,** (Komma) oder **;** (Semikolon) getrennt. Daher der Name. Es können aber auch andere Trennzeichen verwendet werden. Das kann der Fall sein, wenn die Werte – z. B. Kommazahlen – mit den Trennzeichen kollidieren.

Vorteile:

- Simple Struktur, Import und Export kann schnell und einfach programmiert werden
- Leichter Export von relationalen Datenbanken (Tabellen) möglich
- Gutes Verhältnis zwischen Datenstruktur und Nutzdaten

Nachteile:

- Hierachische Darstellung nur über mehrere Dateien und PK/FK-Beziehung möglich.
- Datentypen können nicht explizit definiert werden.
- Keine Darstellung von Key/Value-Paaren →Dictionary

CSV – XML - JSON

XML (e**X**tensible **M**arkup **L**anguage)

```xml
<Personen>
    <Person>
        <Name>Maria Müller</Name>
        <Alter>32</Alter>
        <Stadt>Stuttgart</Stadt>
        <Hobbys>
            <Hobby prio="Prio1">Lesen</Hobby>
            <Hobby prio="Prio2">Radfahren</Hobby>
        </Hobbys>
    </Person>

    <Person>
        <Name>Thomas Becker</Name>
        <Alter>45</Alter>
        <Stadt>München</Stadt>
        <Hobbys>
            <Hobby prio="Prio1">Wandern</Hobby>
            <Hobby prio="Prio2">Kochen</Hobby>
        </Hobbys>
    </Person>
</Personen>
```

Zeile 1 und 21: Es muss immer ein Root-Element vorhanden sein. Die Struktur wird über sog. **Tag**s gebildet (hier blau). Es gibt immer ein Open- und ein Closing-Tag. Innerhalb der Tags können **Attribute** (hier rot) verwendet werden. Diese liefern Zusatzangaben zum Tag. Die Nutzdaten stehen zwischen dem Open- und Closing-Tag (hier schwarz)

Durch die Attribute gibt es die Möglichkeit den Datentyp und weitere Informationen (z. B. Länge des Strings, Zahlenbereich usw.) zu definieren. Das macht das Format sehr flexibel und unversell einsetzbar, denn man kann alle Informationen (Metainfos) in diese Datei mit hineinstecken. Dies erlaubt einen qualifizierten und flexiblen Austausch mit Fremdsystemen.

Häufige Anwendung in:

- Datenaustausch zwischen Fremdsystemen, da die Daten „selbstbeschreibend“ sind. Siehe hierzu XSD/DTD/Schema. → SOAP (Vorgänger von JSON)
- Verwendung in Konfigurationsdateien
- Gestaltung von GUIs (html-like) WPF, Android-Apps

Vorteile:

- Sehr flexibel, weil man die Tags selbst festlegen kann
- Hierarchische Darstellung von Informationen möglich.

Nachteile:

- Wird schnell unübersichtlich.
- Schlechtes Verhältnis zwischen Datenstruktur und Nutzdaten. Großer Overhead bei der Datenübertragung

CSV – XML - JSON

JSON (**J**ava**S**cript **O**bject **N**otation)

Root beginnt mit {}

```json
{
    "personen": [
        {
            "name": "Maria Müller",
            "alter": 32,
            "stadt": "Stuttgart",
            "hobbys": {
                "Prio1": "Lesen",
                "Prio2": "Radfahren"
            }
        },
        {
            "name": "Thomas Becker",
            "alter": 45,
            "stadt": "München",
            "hobbys": {
                "Prio1": "Wandern",
                "Prio2": "Kochen"
            }
        }
    ]
}
```

{ } ist ein Dictionary mit Key:Value-Paaren

[ ] ist ein Array mit Einzelwerden desselben Typs.

Unterscheidung von String und Zahlen (double, int) möglich

Häufige Verwendung:
-REST-APIs (Datenaustausch von unabhängigen Systemen)
-Speichern von Konfigurationsdaten
-Ablage von Objekten (Serialisierung & Deserialisierung) sehr einfach
-In Python: Standard für den Datenaustausch und Datenablage. Es gibt zahlreiche libs.

Vorteil:
-einigermaßen kompakt
-viele Libs vorhanden
-Über Schema auch Validierung und Typisierung möglich

- Verwendung in nicht-relationalen DBs (Mongo)
-

Nachteil:
- nicht geeignet für große Datenmengen, da auch hier Overhead zu groß.

Alternative, Hobbies mit Arrays

```json
{
    "personen": [
        {
            "name": "Maria Müller",
            "alter": 32,
            "stadt": "Stuttgart",
            "hobbys": [
                { "prio": "Prio1", "wert": "Lesen" },
                { "prio": "Prio2", "wert": "Radfahren" }
            ]
        },
        {
            "name": "Thomas Becker",
            "alter": 45,
            "stadt": "München",
            "hobbys": [
                { "prio": "Prio1", "wert": "Wandern" },
                { "prio": "Prio2", "wert": "Kochen" }
            ]
        }
    ]
}
```